C++ Programming

Juha Yli-Hemminki

31.10.2025

SeAMK

SeAMK

if..else

- Like most programming languages C/C++ have if statement to make different branches into program based on given conditions

```
if (condition) {
  // code block that is executed if condition is true
} else {
  // code block that is executed if condition is
  false
}
```

if..else

- Conditions:

  Less than: a < b

  Less than or equal to: a <= b

  Greater than: a > b

  Greater than or equal to: a >= b

  Equal to a == b // beaware! a=b compiles also

  Not Equal to: a != b

if..else

- Logical operators:

Logical and	&&	is true if both sides are true (a<2 && b<10)
Logical or	||	is true if one side is true (a<2 || a>10)
Logical not	!	reverses the result !(a<2 || a>10)

- Example:

```
if (a<2 || a>10) {// if a is less than 2 or greater than 10}
```

while.. do/while

- Similar loop functionality as for loop

```
while(conditions) {
    // this block is repeated as long as condition is true
}
```

- Condition state must be changed inside the block

```
int i = 0;
while (i<5) {
        std::cout << i << std::endl;
        i++; // i iterated by one for each round
}
```

while.. do/while

- In do/while structure code block is executed at least once

```
do {
        // this code block will be run once
} while (false)
```

switch..case

- In switch/case structure one of multiple code blocks is executed

```
switch(value) {
        case 1:
                std::cout  << “Value is 1” << std::endl;
                break;
        case 2:
                std::cout  << “Value is 2” << std::endl;
                break;
        default:
                std::cout << “No value” << std::endl;
}
```

SeAMK

References in C++

- A reference variable is a way to reference to existing variable

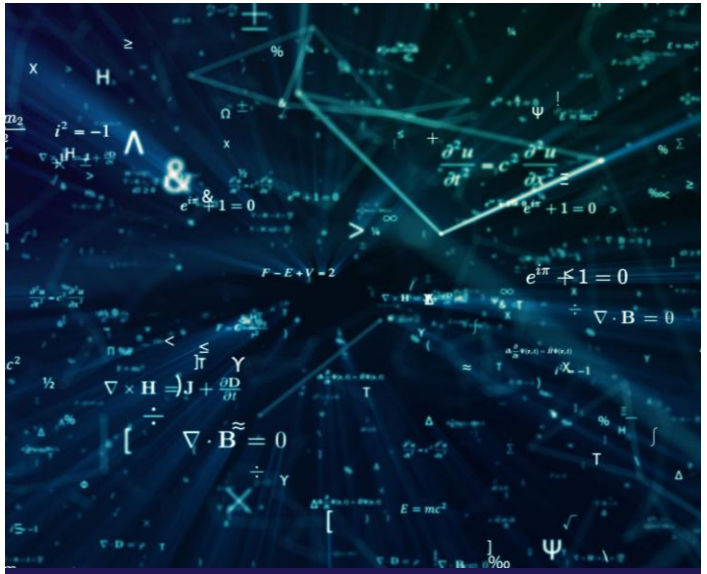
```
int a = 5;      // variable a
int& b = a;     // reference b to variable a
```

- It’s like another name for that existing variable
- That original variable can be used by it’s own variable name or by the reference that refers to that variable

SeAMK

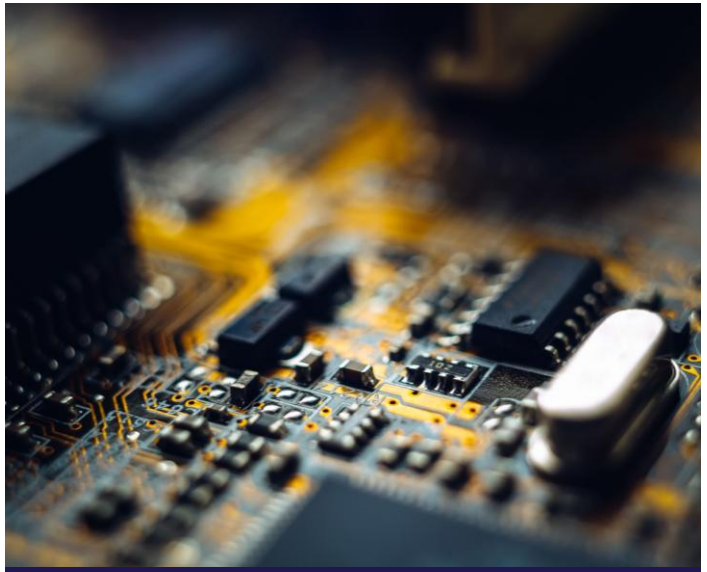
References in C++

- Reference can’t be null. Always refers to an existing variable
- It must be initialized when it is created
- Reference can’t be changed after it is created



Memory address

- When variable is created it gets some place in computer memory
- When we give the value for the variable, value is stored to that place
- That place in computer memory has its own address
- Operator & is also used to find out that address



Memory address

- Example with &:

```
string weekday = “Thursday”;
std::cout << &weekday; // output is something like 004FFDC47
```

- What we got as a printout is the memory address where the variable is stored

Where to keep the memory address: Pointers

- Pointer is variable that stores the memory address

  string* ptr;

- Pointer is created by using asterisk sign * after variable type
- Above is created the pointer named ptr that points to string type variable

Pointers

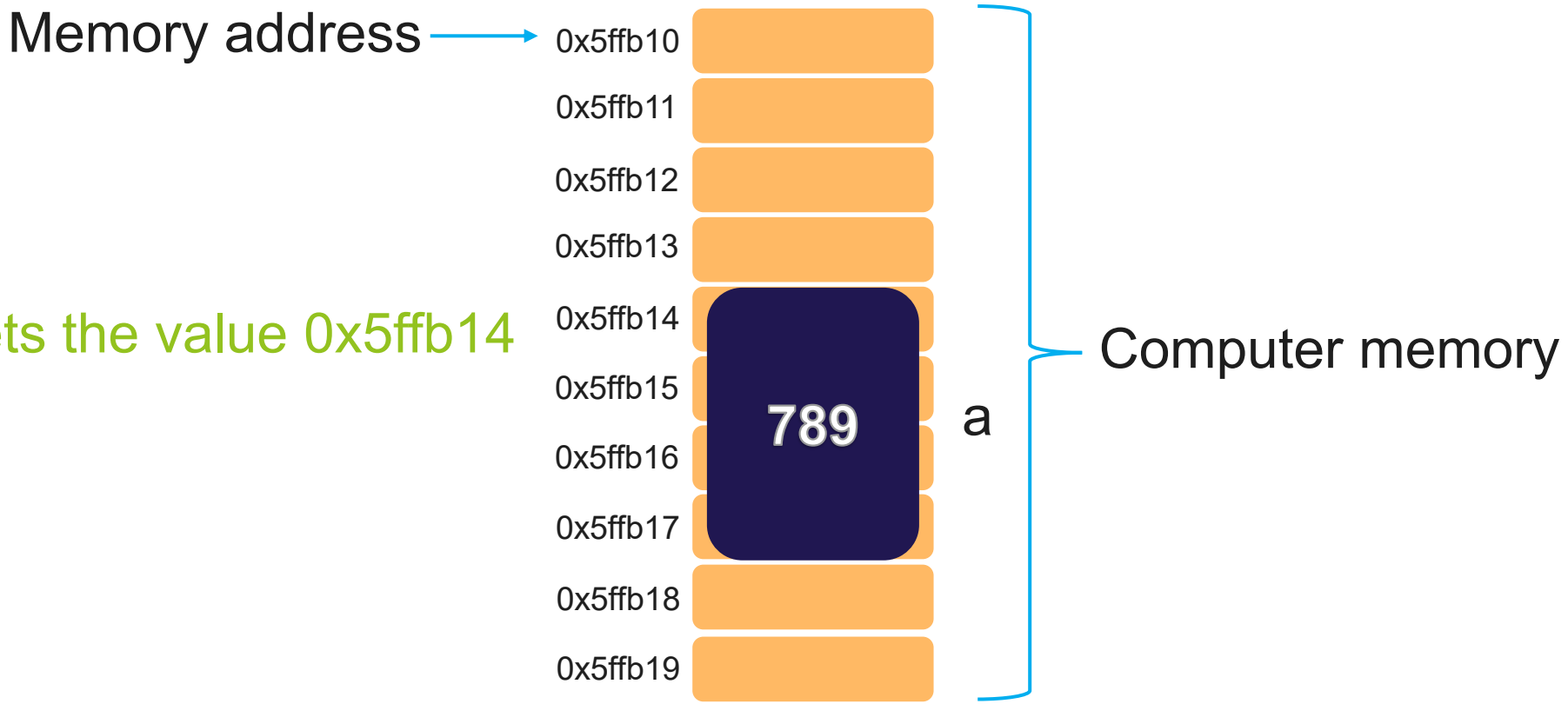
- Example with pointer:

```
string weekday = “Thursday”;
string* ptr = &weekday;
std::cout << ptr; // output is something like 004FFDC47
```

SeAMK

Computer memory

```
int a = 789;
int* ptr = &a; //gets the value 0x5ffb14
```



SeAMK

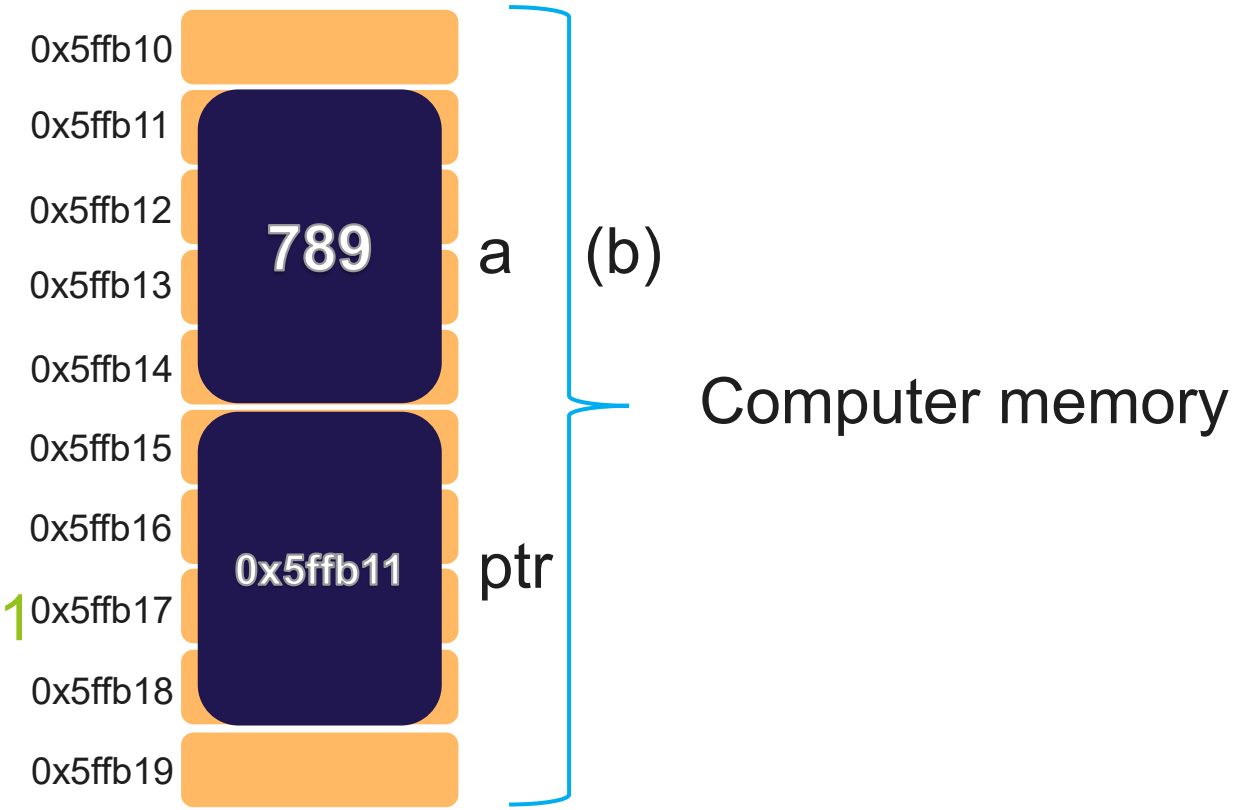
Computer memory

Memory address

```
int a = 789;

Reference:
int& b = a; // alias for a

Pointer:
int* ptr = &a; // gets the value 0x5ffb11
```



SeAMK

SEINÄJOEN AMMATTIKORKEAKOULU

SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES